Digital Systems

Lecture 3

Building a Program

Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Last lecture:

- We understood instructions, and how they affect CPU state.
- We introduced assembly instructions,
- example of tiny subroutine.

💪 **Build and run a program on the BBC Microbit?**

❓ **How do we get our instructions into memory?**

⚠️ **Memory is a bit more complicated than previously implied**

Understanding More about the Microbit

micro:bit

Nordic nRF51822

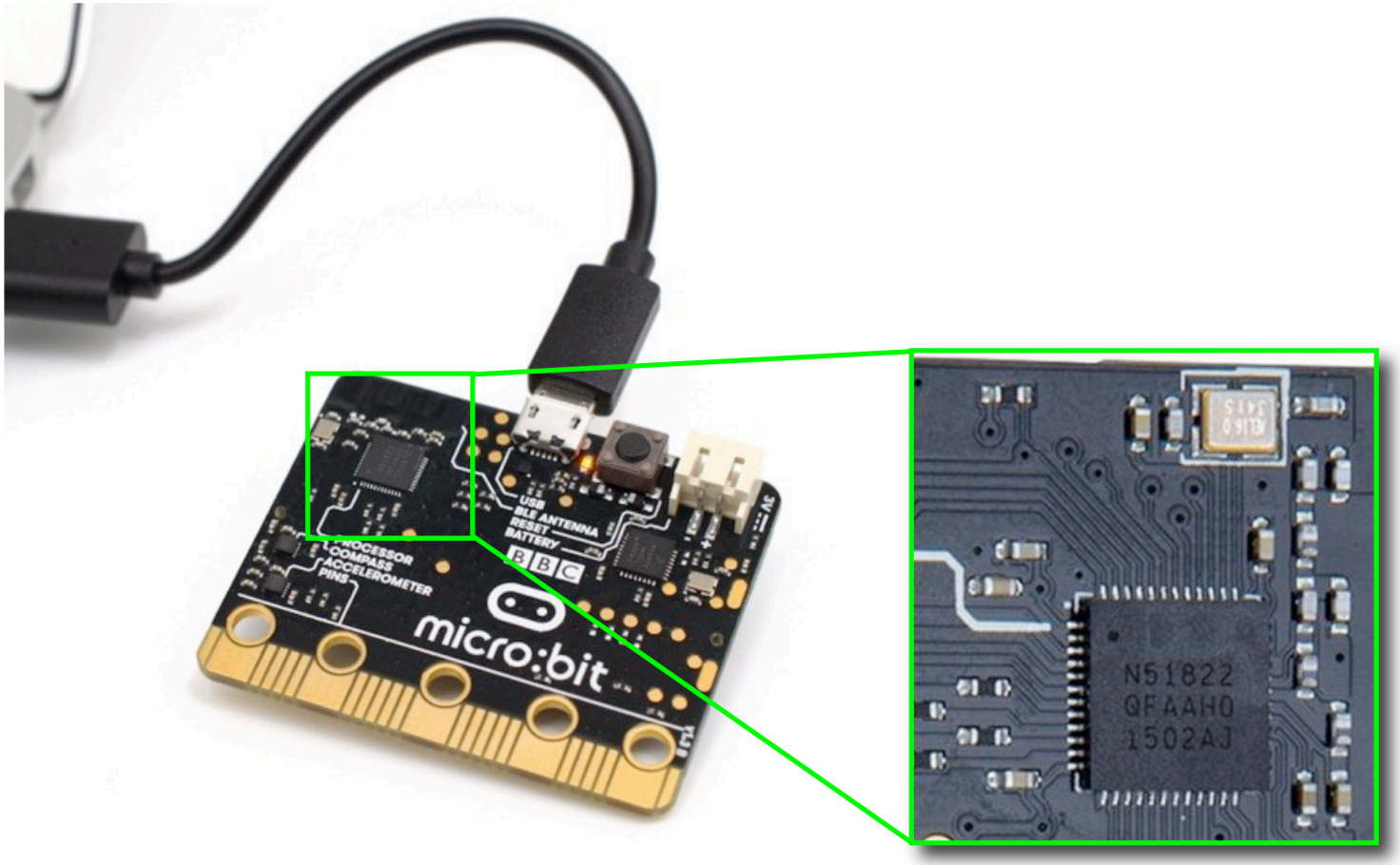
ARM Cortex-M0

- Registers, datapath
- Thumb-based control
- Interrupt controller

- Peripherals: GPIO, UART, I2C
- 16kB RAM, 256kB Flash ROM

- LEDs, buttons via GPIO
- Accelerometer, Magnetometer via I2C
- Second processor – for USB

Understanding More about the Microbit



Transferring a Program to the Microbit

Microbit behaves like USB storage device. To upload program:

- You drag a file, containing the program, to the storage device.
- 2nd processor receives the file over USB
- **Doesn't** store it! So it will disappear just after it is copied!
- Instead, the 2nd procressor's program controls pins on the Nordic chip, and transmits the program .
- The Nordic chip goes into a mode that allows programming the flash.
- Nordic chip stores the program into Flash memory.

Transferring a Program to the Microbit

Microbit behaves like USB storage device. To upload program:

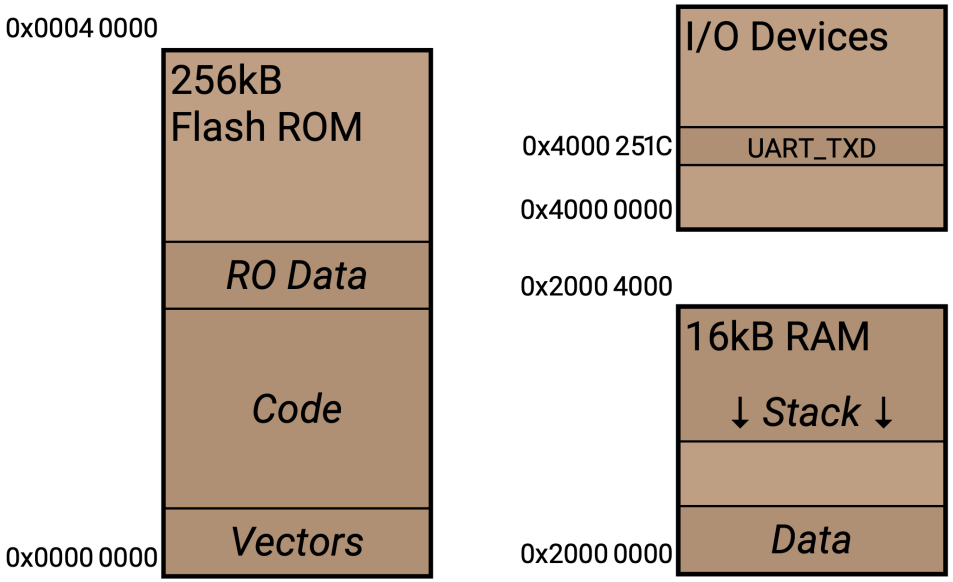
- You drag a file, containing the program, to the storage device.
- 2nd processor receives the file over USB
- **Doesn't** store it! So it will disappear just after it is copied!
- Instead, the 2nd procressor's program controls pins on the Nordic chip, and transmits the program .
- The Nordic chip goes into a mode that allows programming the flash.
- Nordic chip stores the program into Flash memory.

? If there are multiple types of memory, how do you access it?

Memory Layout

Memory is mapped into a single *address space.*

- All memory locations have a 32-bit address
- Different types of memory are just given addresses
- Even peripherals are given memory locations! (UART / GPIO)



- Single address space makes it easy to access all these: can broadly use the same instructions.

Demo: Building and Running Program

```
    .syntax unified      @ Use modern ‘unified’ syntax
    .global func         @ Allow calling func from main
    .text                @ Text segment -- goes into ROM

    .thumb_func
func:                    @ Entry point for function func
@ ----------------
@ Two parameters are in registers r0 and r1

    adds r0, r0, r1      @ One crucial instruction

@ Result is now in register r0
@ ----------------
    bx lr                @ Return to the caller
```

Running Our Subroutine

We need:

- to get our subroutine in the correct location in Memory
- our subroutine to be called (by another program?)
- to be able to observe the results

Show source files of `lab1-asm`

Running Our Subroutine

We need:

- to get our subroutine in the correct location in Memory
- our subroutine to be called (by another program?)
- to be able to observe the results

Show source files of `lab1-asm`

The *only* thing we can do to program this chip, is place instructions in memory.

Running Our Subroutine

We need:

- to get our subroutine in the correct location in Memory
- our subroutine to be called (by another program?)
- to be able to observe the results

Show source files of `lab1-asm`

The *only* thing we can do to program this chip, is place instructions in memory.

So we will need *tools* that convert source files (C and asm)…
to a list of instructions to be placed at specific *addresses*.

Compilation from the Command Line

- `make all` produces `func.hex`, which you can copy to Microbit.
- Look at commands it runs. Notice `arm-none-eabi-as`! **Assembler**!
- `more func.o`: Interpret binary *as* ASCII-encoded text.
- `hexdump func.o`: Print the binary nicely as numbers.
- We notice the machine code we expect! `1840` and `4770`! (Plus metadata)
- `arm-none-eabi-objdump -d func.o`

Demo: Compiling, Assembling, Linking

Assembling our code:

```
$ arm-none-eabi-as add.s -o add.o
```

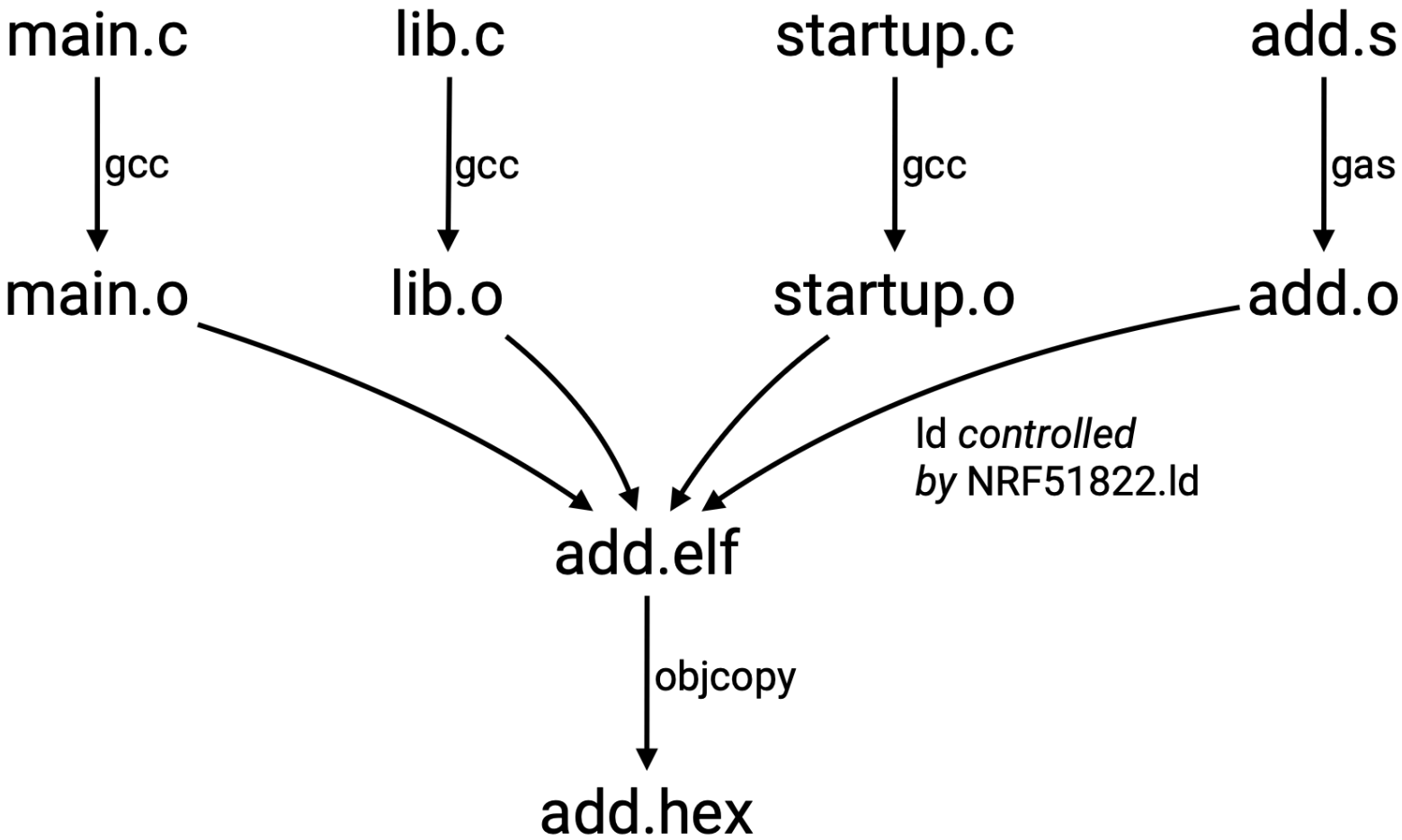
Compiling the parts written in C:

```
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c main.c -o main.o
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c lib.c -o lib.o
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c startup.c -o startup.o
```

Linking it all together:

```
$ arm-none-eabi-ld add.o main.o lib.o startup.o \
     /usr/lib/gcc/arm-none-eabi/5.4.1/armv6-m/libgcc.a \
     -o add.elf -Map add.map -T NRF51822.ld
```

Demo: Compiling, Assembling, Linking



Demo: Programming, Running, Debugging

- `arm-none-eabi-objdump -d func.elf`
  - `func.elf` is **after linking**!
  - What is the difference with earlier?
- Look at Intel Hex format (big-endian vs little-endian)
- Run code. `minicom` for communication.

Debugging

- `layout asm`
- `layout regs`
- `break *func`
- `stepi`

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1

adds r3, r0, r1     @ (a + b) stored in r3

subs r0, r0, r1     @ (a - b) stored in r0

muls r0, r3, r0

bx lr
```